



SwarmOps - Business Overview

SWARM INDUSTRIES
BUSINESS DOCUMENT · AUGUST 2026

Drone Fleet Coordination & Mission Optimization Platform

Prepared for: College leadership and prospective institutional partners **Purpose:** To support a decision on incubation, funding, or institutional backing for SwarmOps as it transitions from a completed academic capstone to an early-stage venture **Date:** August 2026 **Team:** Tony [surname] (team lead), Guy [surname], Harel [surname]

1. Executive Summary

SwarmOps is a software platform that automates the hardest part of operating a drone fleet: deciding, in real time, which drone should fly which mission, in what order, along which route - while guaranteeing that no drone runs out of battery mid-flight.

SwarmOps is not a concept or a slide deck. It is a working distributed system, deployed and demonstrable today on AWS: a real optimization engine solving a variant of the Vehicle Routing Problem with Time Windows (VRPTW), an event-driven architecture that re-plans the fleet within seconds of a disruption, a live operational map, and a fully automated, production-grade delivery pipeline of the kind used by mature software companies.

The current version coordinates a simulated fleet - a deliberate scope decision of the academic project, not a technical limitation. The coordination layer is hardware-agnostic by design, and integration with real drone hardware is the first, well-defined step of commercialization.

We are seeking institutional backing - incubation, mentorship, industry introductions, and pre-seed support - to take SwarmOps from a validated engineering foundation to a piloted commercial product within 12 months.

2. The Problem

Drone fleets are growing faster than the tools used to run them.

A human dispatcher can manage a handful of drones. Beyond that, the coordination problem becomes computationally intractable for a person: battery limits that depend on distance *and* payload weight, payload-to-mission compatibility, mission priorities and deadlines, no-fly zones, charging logistics, and mid-mission disruptions (a drone drains faster than predicted, an urgent mission arrives, a drone goes offline) - all interacting at once.

The operational consequences are consistent across sectors:

- **Wasted flight time and low utilization** of expensive aircraft, because assignments are made by intuition rather than optimization.
- **Missed deadlines on high-priority missions**, because a human cannot re-plan the whole fleet every time conditions change.

- **Stranded aircraft**, because battery feasibility is estimated rather than computed.
- **Duplicate coverage and an inaccurate operational picture**, because without central coordination two teams can sweep the same ground and report it differently.
- **No single operational picture**, because fleet state lives across separate tools, spreadsheets, and radio calls.

Why we built this - this is not a theoretical scenario for us. On 7 October 2023, during the fighting in the kibbutzim, two teams from the same company launched drones at the same time. With no coordination between them, both swept the same sector - one team identified hostiles, the other reported a friendly force in the same place. The contradiction in the situational picture delayed a critical operational decision. That is the problem SwarmOps was built to solve.

Existing tooling is largely built by drone *manufacturers* and is drone-centric and vendor-locked: it flies one aircraft well, but it does not answer the fleet-level question - *who should do what, and can they actually complete it?* That coordination layer is the gap SwarmOps fills.

This problem is directly relevant to defense and security logistics - resupply tasking, patrol and surveillance coverage, inspection rounds over dispersed infrastructure - as well as to civilian sectors such as energy and infrastructure inspection, agriculture, and last-mile delivery.

3. The Product - What Exists Today

Everything in this section is built, deployed, and demonstrable live. Nothing here is planned or projected.

3.1 Optimization engine

The core of SwarmOps is a mission-assignment and routing engine that treats the fleet, not the individual drone, as the unit of operation:

- **Optimal assignment** of drones to missions (Hungarian algorithm / weighted bipartite matching), minimizing total travel while penalizing lateness on urgent missions - benchmarked against a greedy baseline.
- **Per-drone multi-stop routing** (Google OR-Tools) for drones carrying several missions.
- **Battery feasibility as a first-class constraint:** the engine simulates battery drain along every candidate route - as a function of distance flown and payload weight, not a fixed capacity - and rejects any plan that would strand a drone, automatically inserting charging-station stops where needed.
- **Conflict resolution:** when two missions compete for the same drone, the engine resolves in favor of priority and deadline, and flags what it could not resolve to a human dispatcher.

3.2 Real-time, event-driven re-planning

Telemetry from the fleet (position, battery, status) streams through a message bus into the planning engine. When reality diverges from the plan - a new urgent mission, an abnormal battery drain, a drone dropping offline - the system re-solves the affected portion of the schedule automatically, within seconds, without a human pressing "re-plan."

3.3 Operational interface

A web-based control room: a live map showing every drone, mission, planned route, and no-fly zone, updating in real time; a mission board for creating and tracking missions; fleet inventory with battery and maintenance state; and operational dashboards (fleet utilization, mission completion rate, solve time, conflict rate).

3.4 Production-grade engineering - not a student prototype

SwarmOps was built end-to-end to the operating standard of a commercial software company, and this is a core asset of the venture:

- **Architecture:** eight independent microservices (authentication, fleet, missions, planning, telemetry, notifications, gateway, fleet simulator) communicating over an event bus (RabbitMQ), each independently deployable and scalable.
- **Cloud:** running on AWS (Kubernetes/EKS), with all infrastructure defined as code (Terraform) - the entire environment can be destroyed and rebuilt from scratch on demand, which also keeps cloud spend controlled.
- **Delivery:** fully automated CI/CD (GitHub Actions, keyless OIDC authentication to AWS) with GitOps-based deployment (Argo CD) - every deployment is a Git commit; no one touches the cluster by hand. The highest-risk service (the planning engine) ships through automated canary rollouts with automatic rollback.
- **Observability:** full metrics and structured-log coverage (Prometheus, Grafana, Loki) with live operational dashboards.

For an evaluator, the significance is this: the distance from "capstone" to "deployable product" is unusually short, because the operational maturity that startups typically retrofit in year two already exists.

3.5 Current scope - stated plainly

The current fleet is **simulated**: a high-fidelity simulator flies each drone along its assigned route at realistic speeds with randomized variance (wind, sensor noise), generating the genuine telemetry stream the re-planner reacts to. Real hardware integration was deliberately out of scope for the capstone. Because every drone-facing interface in the system is already abstracted (drones are producers of telemetry and consumers of routes), integrating real aircraft - e.g., via the MAVLink protocol, the de-facto standard supported by most commercial autopilots - is an engineering task with a clear path, not a research risk. It is Phase 1 of the roadmap below.

4. Market Opportunity

(Quantitative market sizing to be added from third-party sources in the full business plan; the figures below are deliberately omitted rather than estimated.)

Three converging trends make fleet-level coordination software valuable now:

1. **Fleets are scaling.** Drone operations in defense, inspection, and delivery are moving from single-pilot, single-drone operations to multi-drone fleets - and regulation worldwide is

progressively enabling beyond-visual-line-of-sight (BVLOS) operations, which multiplies the number of aircraft one operator is responsible for and makes manual dispatch untenable.

2. **Hardware is commoditizing; coordination is not.** The differentiating layer is shifting from the aircraft to the software that decides what the fleet does. Manufacturer software does not solve cross-fleet optimization, and it locks operators into one vendor.
3. **Israel is a natural first market:** one of the world's densest ecosystems of drone manufacturers, defense integrators, and security operators - a rich pool of pilot partners, design partners, and early customers within direct reach of the college's industry network.

Target segments, in order of approach:

Segment	Use case	Why early
Defense & security logistics	Resupply tasking, patrol/surveillance coverage planning, inspection rounds	Largest fleets, strongest need for centralized coordination, strong local ecosystem
Infrastructure & energy inspection	Power lines, pipelines, solar fields - recurring, route-heavy missions	Scheduled, repeatable missions map directly onto the optimizer
Agriculture	Survey and spraying coordination across large areas	Battery/payload constraints are acute; seasonal but high-volume
Logistics & delivery	Last-mile and site-to-site delivery	Longer-term; regulatory timeline is the gate, not the technology

Positioning: SwarmOps does not build drones and does not compete with manufacturers. It is the vendor-neutral coordination layer - the operations brain - above whatever aircraft an organization already owns.

5. Differentiation

- **Optimization-first.** Assignment and routing are the core of the product, not a scheduling screen bolted onto a flight app. The engine produces measurably better plans than manual/greedy assignment and proves battery feasibility for every route it emits.
- **Event-driven by architecture.** Re-planning is triggered by live telemetry in seconds. This is an architectural property, not a feature toggle - retrofitting it onto a request/response system is a rewrite.
- **Vendor-neutral and hardware-agnostic.** The fleet interface is abstract by design; the platform's value grows with fleet heterogeneity, which is exactly where manufacturer tools fail.
- **Operationally mature from day one.** Automated, auditable, canary-gated deployment and full observability - properties that materially de-risk early pilots with security-sensitive customers.

6. Status & Traction

- Capstone completed and presented; the full system is deployed on AWS and demonstrable live, end to end, on demand.

- In simulation runs, the planner produces zero battery-infeasible routes, and event-driven re-planning responds to fleet disruptions within seconds - the project's success criteria were defined up front and met.
- Following the presentation, college management expressed interest in supporting the project's growth into a startup - the immediate cause of this document.

7. Roadmap - Capstone to Company (12 months)

Phase 1 (months 0–3): Real aircraft. Incorporate. Build the hardware abstraction layer and integrate a first physical drone via MAVLink; demonstrate the full loop (assignment → route → live telemetry → re-plan) on real hardware. Begin regulatory groundwork with the Civil Aviation Authority of Israel.

Phase 2 (months 3–6): First pilot. A scoped pilot with one design partner - infrastructure inspection or campus/site security are the natural candidates - running a small mixed fleet on recurring missions. Success criteria defined with the partner up front (utilization, planning time saved, zero battery incidents).

Phase 3 (months 6–12): Productization. Multi-organization support (multi-tenancy), operator onboarding, pricing model validated against the pilot; convert the pilot to a paying reference customer; raise a seed round on the back of it.

Items that were explicit non-goals of the capstone - real hardware, multi-tenancy - appear here as roadmap phases. Nothing on this roadmap contradicts what has been built; it extends it.

8. Team

Three engineers who designed, built, and operate the entire system - algorithms, services, frontend, cloud infrastructure, and delivery pipeline:

- **Tony [surname]** - team lead. [1–2 lines: focus areas, e.g., CI/CD pipeline, cloud infrastructure, planning engine.]
- **Guy [surname]** - [1–2 lines: focus areas.]
- **Harel [surname]** - [1–2 lines: focus areas, e.g., GitOps, progressive delivery, observability.]

The team has operated as a distributed engineering organization - independent repositories, contract-first interfaces, protected mainlines, and automated deployment - which is precisely the working model the company will scale with.

9. The Ask

To execute Phase 1–2 of the roadmap, we are requesting the college's support in the following forms:

1. **Incubation** - workspace and formal affiliation with the college's entrepreneurship program.
2. **Mentorship** - business development and defense/aerospace industry guidance.

3. **Introductions** - to the college's industry network: drone operators, defense integrators, and infrastructure companies, for design-partner and pilot conversations.

4. **Pre-seed support** - [amount] to cover hardware for Phase 1 (2–3 development drones and autopilot kits), cloud costs, and regulatory/legal setup.

We would welcome the opportunity to present a live demonstration of the deployed system to leadership and to any prospective partner.

Contact: Tony [surname] - [email] · [phone]